# Lab 7 — Usability and Accessibility Evaluation

Sample prompts and code · A5 · LO3

## prompt.txt

```
Goal: test record, prioritised findings and before/after design checkpoint
Audience: Adults using a community wellness service, including users with varied digital confidence and
  accessibility needs.
Context: Evaluate the prototype with a realistic task, heuristic review and accessibility checks, then
  perform one evidence-led iteration.
Required content: Use realistic service names, appointment times, success, empty, loading, permission and
  error states.
Design system: Use the supplied tokens and component rules. Prefer clear labels, visible focus, 44px minimum
  targets and strong contrast.
Constraints: Mobile-first; no lorem ipsum; one dominant primary action per screen; preserve user input on
  recoverable errors.
Acceptance: Metrics defined before testing; Findings cite evidence; Accessibility covers states; Retest uses
  the same task
Output: Create the named design in Claude Design, save a version checkpoint and export or share the evidence
  requested in the README.
```

## test-script.md

```
# Test script

Task: Book a counselling appointment for the earliest available afternoon time.

Metrics: task success, time, critical errors, help requests, confidence 1–5, keyboard completion.

Facilitator: Do not explain the interface. Ask neutral follow-ups after the task.
```

## results.csv

```
participant,task_success,time_seconds,critical_errors,help_requests,confidence,observation
P1,yes,72,0,0,4,Noticed conflict and recovered
P2,no,118,1,1,2,Missed error message
P3,yes,81,0,0,4,Used keyboard successfully
```

## accessibility-check.js

```
// Run in the browser console on the prototype.
const issues = [];
document.querySelectorAll('button, input, select, a[href]').forEach((el) => {
  const style = getComputedStyle(el);
  const rect = el.getBoundingClientRect();
  if (rect.width < 44 || rect.height < 44) issues.push({type:'target-size',
    element:el.outerHTML.slice(0,80)});
  if (style.outlineStyle === 'none') issues.push({type:'verify-focus-visible',
    element:el.outerHTML.slice(0,80)});
});
document.querySelectorAll('input,select').forEach((el) => {
  if (!el.labels?.length) issues.push({type:'missing-label', id:el.id});
});
console.table(issues);
```

## iteration-log.md

```
# Iteration log

Evidence:
Hypothesis:
Change:
Expected result:
Retest result:
Decision:
```